Tensors

N-dimensional arrays with GPU support and autograd. The atom of PyTorch.

🧪 Shape Playground

Pick a creation function and shape — see what tensor you'd get.

Function: torch.zeros Shape: 2, 3, 4 dtype: torch.float32

```
>>> torch.zeros(2, 3, 4, dtype=torch.float32)
# zeros() — filled with 0
tensor([[[0., ...]]])
# shape: torch.Size([2, 3, 4])
# numel: 24
```

2 × 3 × 4

Creation

torch.tensor · from data

Copy

Build a tensor from a Python list, NumPy array, or scalar.

```
import torch

x = torch.tensor([[1, 2], [3, 4]])
y = torch.tensor(3.14, dtype=torch.float64)
z = torch.from_numpy(np_array)  # shares memory
```

zeros / ones / full

Copy

Filled tensors of a given shape.

```
torch.zeros(2, 3)
torch.ones(3, 3)
torch.full((2, 2), 3.14)
torch.empty(5)  # uninitialized
torch.zeros_like(x)  # match shape+dtype+device
```

rand / randn / randint

Copy

Random tensors. `rand` is uniform [0,1), `randn` is standard normal.

```
torch.rand(2, 3)                # uniform [0,1)
torch.randn(3, 3)               # N(0,1)
torch.randint(0, 10, (2, 2))      # int in [low, high)
torch.manual_seed(42)
```

arange / linspace

Copy

1-D sequences.

```
torch.arange(0, 10, 2)    # [0,2,4,6,8]
torch.linspace(0, 1, 5)    # 5 points 0..1
torch.logspace(-2, 2, 5)
```

dtype & casting

Copy

Change a tensor's data type.

```
x.float()    # float32
x.double()   # float64
x.long()     # int64
x.bool()
x.to(torch.float16)
x.type_as(other)
```

shape inspection

Copy

Read tensor metadata.

```
x.shape      # torch.Size([2, 3])
x.size(0)    # 2
x.ndim       # 2
x.numel()    # total elements
x.dtype      # torch.float32
x.device     # cuda:0 / cpu
x.item()     # Python scalar (1-elem only)
x.stride()    # bytes between elems per dim
```

```
x.is_contiguous()
x.is_leaf     # end of autograd graph?
```

*_like family

Copy

Match an existing tensor's shape, dtype, device.

```
torch.zeros_like(x)
torch.ones_like(x)
torch.empty_like(x)
torch.full_like(x, 3.14)
torch.rand_like(x)   torch.randn_like(x)
torch.randint_like(x, 0, 10)
x.new_zeros(3, 4)   # same dtype/device, new shape
```

eye / diag / triu / tril

Copy

Identity / diagonal / triangular matrices.

```
torch.eye(3)                 # identity 3×3
torch.diag(v)                # 1-D → 2-D diag
torch.diagflat(v)
torch.diagonal(M, offset=0)
torch.triu(M, diagonal=0)    # upper
torch.tril(M, diagonal=0)    # lower
```

meshgrid / cartesian_prod

Copy

Build coordinate grids.

```
x = torch.linspace(-1, 1, 5)
y = torch.linspace(-1, 1, 5)
gx, gy = torch.meshgrid(x, y, indexing="ij")

torch.cartesian_prod(x, y)   # all pairs
torch.combinations(x, r=2)
```

complex tensors

Copy

Real ↔ complex arithmetic.

```
c = torch.complex(real, imag)
c = torch.polar(abs_, angle)
c.real     c.imag
c.conj()   c.abs()   c.angle()
```

```
torch.view_as_complex(x)    # (..., 2) → complex
torch.view_as_real(c)       # complex → (..., 2)
```

sparse tensors

Copy

COO and CSR sparse formats.

```
idx = torch.tensor([[0, 1], [2, 0]])
val = torch.tensor([3., 4.])
s = torch.sparse_coo_tensor(idx, val, (2, 3))
s.to_dense()
x.to_sparse()
x.to_sparse_csr()
```

numpy / list interop

Copy

Bridge to Python, NumPy, raw memory.

```
torch.from_numpy(arr)   # shares memory!
torch.as_tensor(data)    # avoids copy when possible
x.numpy()               # CPU only
x.tolist()
torch.frombuffer(buf, dtype=torch.float32)
```

Random Sampling

distributions

Copy

Sample from common distributions.

```
torch.bernoulli(probs)
torch.multinomial(probs, num_samples=5)
torch.normal(mean=0.0, std=1.0, size=(3, 3))
torch.poisson(rates)
torch.randperm(10)              # shuffled 0..9
```

torch.distributions

Copy

Reparameterized sampling + log-probabilities.

```
from torch.distributions import Normal, Categorical

d = Normal(loc=0., scale=1.)
x = d.rsample((100,))    # reparameterized
lp = d.log_prob(x)
Categorical(logits=l).sample()
```

seeds & generators

Copy

Control randomness.

```
torch.manual_seed(42)
torch.cuda.manual_seed_all(42)

g = torch.Generator(device="cpu")
g.manual_seed(0)
torch.randn(3, generator=g)
torch.initial_seed()
torch.get_rng_state()
```